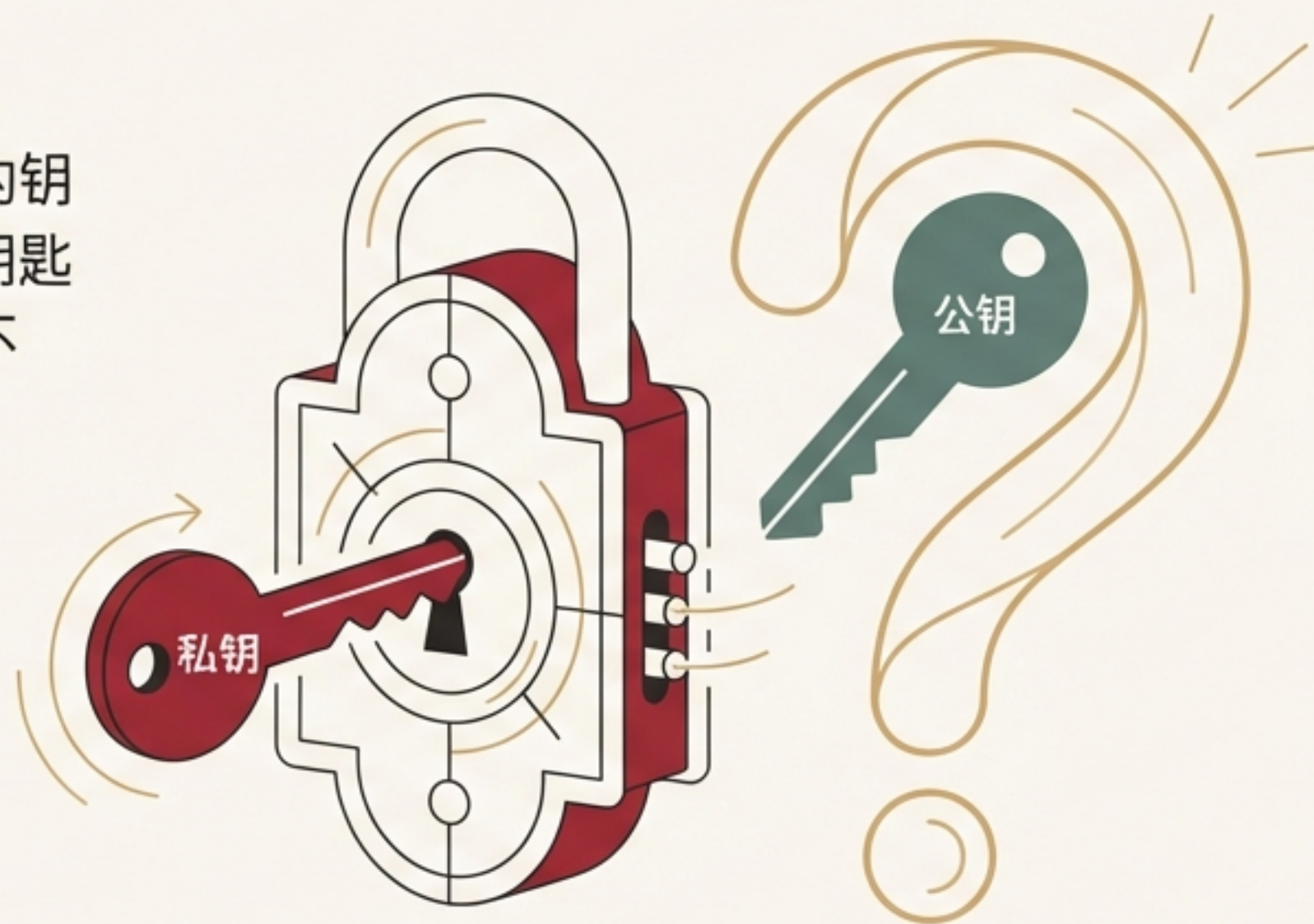


一个似乎矛盾的问题：公钥如何“解开”私钥的签名？

在物理世界中，一把私密的钥匙用来上锁，一把公开的钥匙用来开锁，这听起来完全不合逻辑。



但在数字签名的世界里，这正是发生的事情：用**私钥 (Private Key)**创建的签名，却需要用**公钥 (Public Key)**来验证。

这背后隐藏着怎样的数学魔法？

故事的背景：非对称加密的世界

要解开这个谜题，我们首先需要了解 **非对称加密（Asymmetric Cryptography）**。
这是一种使用一对在数学上相关联的密钥的系统：

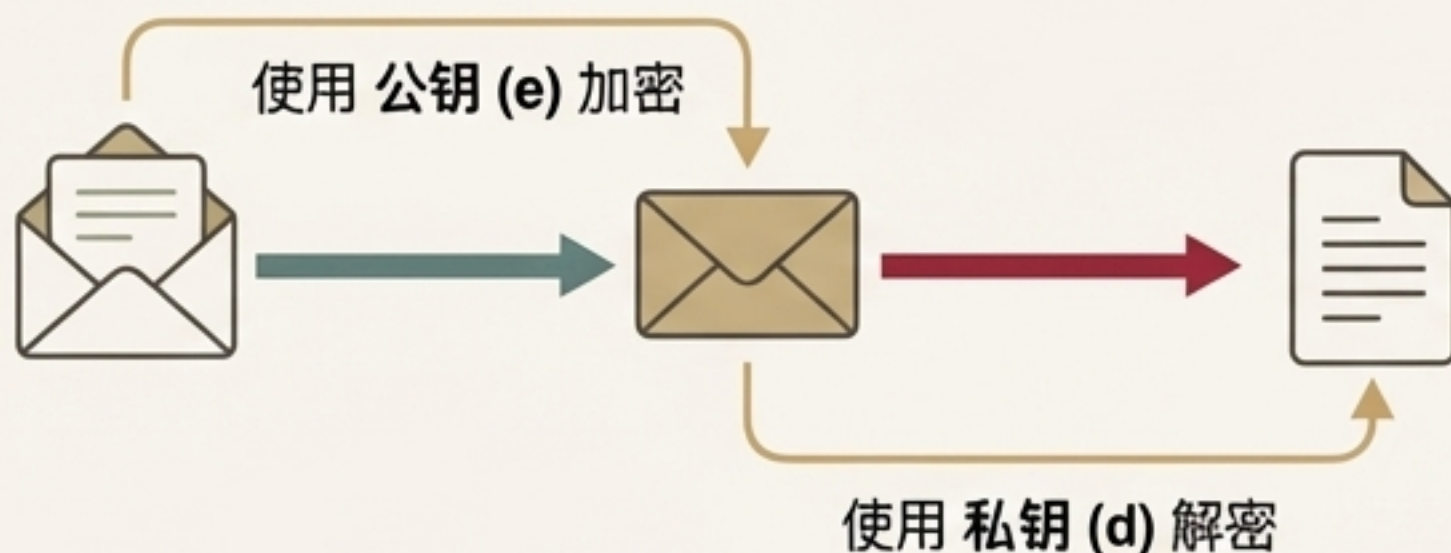


这两个密钥成对工作，一个用于加密或签名，另一个则用于解密或验证。

密钥的双重身份：加密与签名

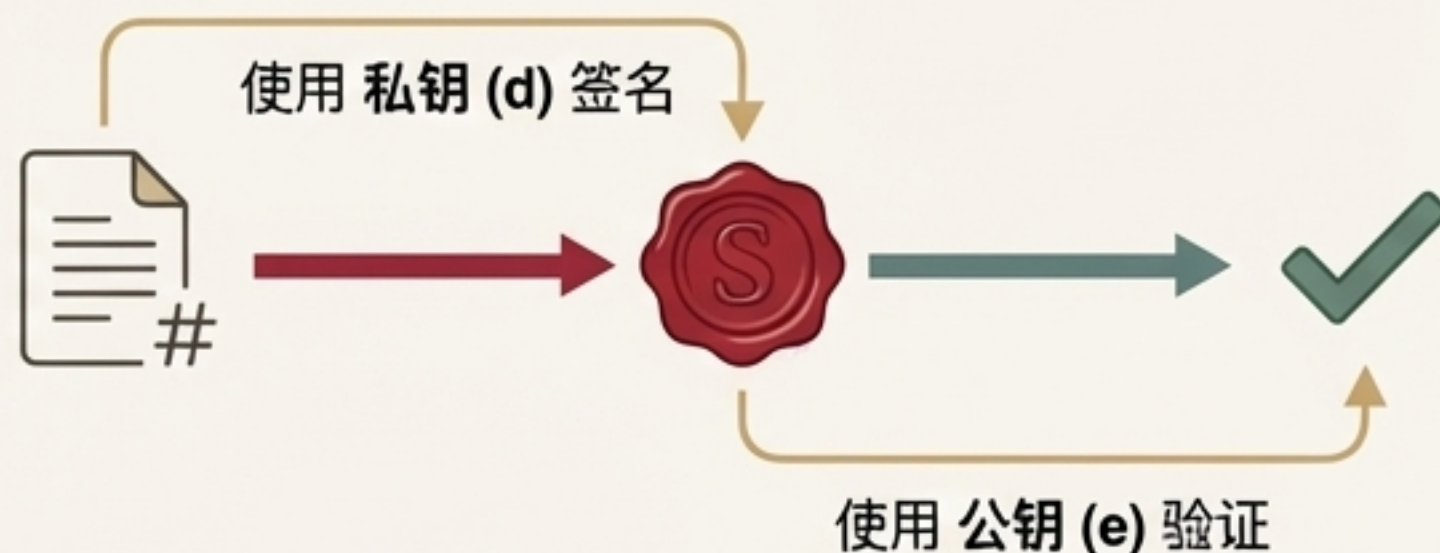
公钥和私钥在不同场景下扮演着相反的角色。理解这一点至关重要。

数据加密 保证机密性



- **加密：**使用 **公钥 (e)** 加密信息。
- **解密：**只有对应的 **私钥 (d)** 才能解密。
- **目标：**只有预期的接收者能阅读信息。

数字签名 保证真实性



- **签名：**使用 **私钥 (d)** “加密”数据的哈希值。
- **验证：**使用 **公钥 (e)** “解密”签名以验证。
- **目标：**任何人都可以验证信息来自你，且未被篡改。

追踪线索 (1)：签名的诞生

假设 Alice 想发送一份带签名的文件。她的操作步骤如下：



1. 原始消息 (M)

Alice 写好一份消息。

2. 计算哈希 (h)

她对消息进行哈希处理，得到一个独一无二的“数字指纹” $h = H(M)$ 。

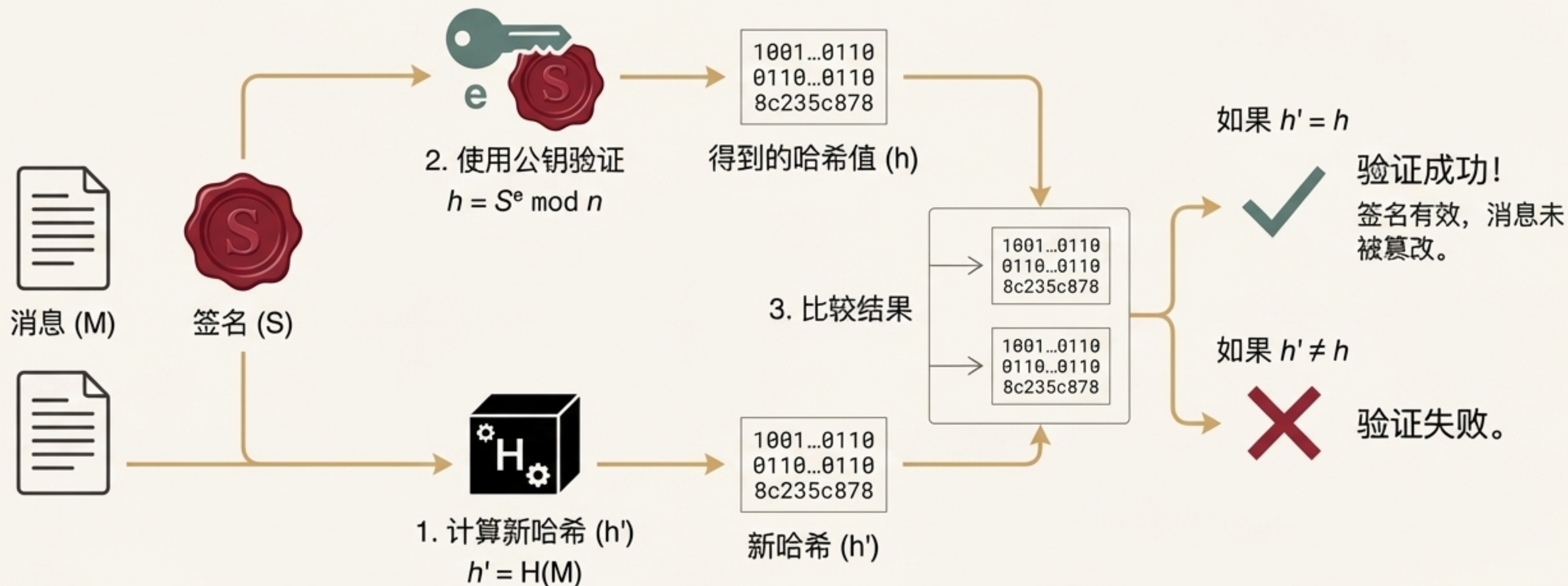
3. 使用私钥签名 (S)

Alice 用她的私钥 (d) 对哈希值 h 进行加密，生成数字签名 S 。

$$S = h^d \bmod n$$

追踪线索（2）：验证的时刻

Bob 收到了 Alice 的消息 (M) 和签名 (S)。他需要用 Alice 的 **公钥 (e)** 来验证。



揭开谜底：神奇的“模乘法逆元”

公钥和私钥之所以能如此配对工作，是因为它们在数学上是一种特殊关系：**互为模乘法逆元 (Modular Multiplicative Inverse)**。

简单来说，逆元是指能够“反转”或“抵消”某个运算的元素。

在 RSA 算法中，公钥 (e) 和私钥 (d) 的关系由以下公式定义：


$$e \cdot d \equiv 1 \pmod{\varphi(n)}$$

这个公式意味着 (e) 和 (d) 在模 $\varphi(n)$ 的世界里互为乘法逆元。 $\varphi(n)$ 是欧拉函数。

理解逆元：一个“时钟”的例子

让我们用一个简单的例子来理解逆元。

****问题**** 在一个只有7个小时的时钟上（即模7的世界），什么数字乘以3的结果是1？

我们需要找到一个数 x ，使得 $3 \times x \equiv 1 \pmod{7}$ 。

****解答****：

$$3 \times 1 = 3 \equiv 3 \pmod{7}$$

$$3 \times 2 = 6 \equiv 6 \pmod{7}$$

$$3 \times 3 = 9 \equiv 2 \pmod{7}$$

$$3 \times 4 = 12 \equiv 5 \pmod{7}$$

$$3 \times 5 = 15 \equiv 1 \pmod{7}$$

答案是5。因此，在模7的体系下，5是3的乘法逆元。

****结论****：就像5可以“抵消”3的乘法效果一样，公钥（e）也能“抵消”私钥（d）的加密效果。



谜题破解：当数学公式开始“对话”

让我们回顾一下签名和验证的数学过程，看看逆元是如何发挥作用的。

Step 1: 签名 (使用私钥 d)

$$\text{Signature} = (\text{Hash})^d \bmod n$$

Step 2: 验证 (使用公钥 e)

$$\text{Verified Hash} = (\text{Signature})^e \bmod n$$

Step 3: 将第一步代入第二步

$$\text{Verified Hash} = ((\text{Hash})^d)^e \bmod n$$

关键时刻 (The Key Moment)

根据指数规则, $((\text{Hash})^d)^e$ 等于 $(\text{Hash})^{d \cdot e}$ 。

$$\text{Verified Hash} = (\text{Hash})^{d \cdot e} \bmod n$$

因为 $e \cdot d \equiv 1 \pmod{\varphi(n)}$, 这使得整个指数运算在模 n 的作用下被“抵消”。

最终结果 (Final Result)

$$\text{Verified Hash} = \text{Hash}$$

答案揭晓：公钥为何能验证签名

我们最初的问题是：为什么公钥可以“解密”由私钥创建的签名？

答案：

因为公钥（ e ）和私钥（ d ）
在数学上是 **模乘法逆元**。

签名过程是用私钥（ d ）对信息指纹进行一次数学运算（加密）。

验证过程则是用公钥（ e ）进行一次**逆运算**（解密），从而完美地还原出原始的信息指纹。

这种可逆性确保了只有私钥的真正持有者才能生成有效的签名。



超越数学：逆元关系构建的数字信任

私钥和公钥之间的逆元关系不仅是一个巧妙的数学技巧，
它更是现代数字安全的基石。

身份认证 (Authentication)

只有你（私钥持有者）才能生成你的
签名。



数据完整性 (Integrity)

任何对消息的微小篡改都会导致哈希
值不匹配，从而使签名验证失败。

正是这种优雅而稳固的数学关系，让我们能够在数字世界中建立信任。